

A scheduled-time feature from projected scores occupancy fractions, the weighted survival function, and where the kernel goes

20 August 2026

The question, as posed

Use the mining / stripping capacity to define the fraction of a time period that mining each block would take, for the sequence of blocks passed to the transformer. Given those fractions, build a feature t that respects precedence in the projected score, following the rule

$$t_j = \tau_j + t_i \quad \text{if} \quad s^*(x_i) > s^*(x_j),$$

with i, j block indices and s^* the projected score. Consider whether the kernel interpolation technique can build t from the projected scores.

Grounding note. `soft_rank.py:soft_npv` already computes $W_r[i] = \sum_j w_{r,j} \sigma((s_j - s_i)/\tau)$, the resource consumed before block i . That is the object below, up to a division by capacity. What is new is exposing it as a *feature* and building it in $O(n \log n)$ rather than $O(n^2)$.

1. The occupancy fraction

Let block j have consumption $w_{r,j}$ on resource r and let C_r be the per-period capacity of that resource. In `capacity_cuts.make_resource` the axes are already

$$\text{mining} = \text{tonnage}, \quad \text{processing} = \text{tonnage} \cdot \mathbf{1}[\text{ore}], \quad \text{stripping} = \text{tonnage} \cdot \mathbf{1}[\text{waste}].$$

The fraction of a period that block j occupies is

$$\tau_j = \max_r \frac{w_{r,j}}{C_r}$$

The maximum, not the mining term alone: a period ends when the *first* resource binds, so charging only mining tonnage silently assumes the mill never binds.

Note that τ is not the stripping ratio. The ratio $w_{\text{waste},j}/w_{\text{ore},j}$ enters only through *which* axis attains the maximum. A separate stripping cap is redundant unless waste handling is independently constrained, since mining = processing + stripping leaves only two of the three axes free.

With time measured in periods, $\sum_j \tau_j$ is the horizon needed to mine everything, and a T -period schedule of constant capacity is feasible only if $\sum_j \tau_j \leq T$.

2. The recursion telescopes into a weighted survival function

The rule $t_j = \tau_j + t_i$ is correct only when i is the *immediate* predecessor of j in score order. Applied to every higher-scoring i it double-counts. Write $\pi(j)$ for the block whose score is the smallest one exceeding s_j^* ; then $t_j = \tau_j + t_{\pi(j)}$ telescopes along the score order to the closed form

$$t_j = \tau_j + G(s_j^*), \quad G(s) := \sum_i \tau_i \mathbf{1}[s_i^* > s]$$

So t_j is not a pairwise object at all. It is the block’s own occupancy plus the τ -**weighted survival function of the score distribution, evaluated at the block’s own score**. Equivalently, with $\mu = \sum_i \tau_i \delta_{s_i^*}$ the score-mass measure, $G(s) = \mu((s, \infty))$.

G is scalar-in, scalar-out, non-increasing, and piecewise constant with n jumps of height τ_i . Everything cheap below follows from that one fact. Here $G(s_j^*)$ is the *start* time of block j and t_j its *completion* time.

Proposition 1 (precedence is inherited, at no cost). *If $(i \rightarrow j) \in E$ and the projection has delivered $s_i^* \geq s_j^*$, then $G(s_i^*) \leq G(s_j^*)$, i.e. block j never starts before block i .*

Proof. G is non-increasing and $s_i^* \geq s_j^*$. □

No extra constraint is required: precedence in score space transfers to the time feature for free. But the inheritance is *weak*. Exact ties give equal start times, which the resource constraint cannot honour. Ties therefore need breaking, either lexicographically ($+\varepsilon \cdot \text{rank}$) or by splitting the tied mass,

$$t_j = \tau_j + G(s_j^*) + \frac{1}{2} \sum_i \tau_i \mathbf{1}[s_i^* = s_j^*].$$

3. Kernel interpolation — yes, but in score space

Smooth the step function with a kernel CDF Φ of bandwidth h :

$$G_h(s) = \sum_i \tau_i \Phi\left(\frac{s_i^* - s}{h}\right).$$

With Φ logistic this is *identically* the pairwise sigmoid already inside `soft_npv`. The soft-rank you have and the kernel-smoothed survival function are the same operator seen from two sides. The existing code is $O(n^2)$ only because it evaluates G_h separately at each s_j^* , when G_h is a single univariate curve.

The interpolation version:

1. choose $m \ll n$ anchor knots $\sigma_1 > \sigma_2 > \dots > \sigma_m$ at τ -weighted score quantiles, so knots are dense where score mass is dense;
2. evaluate G_h exactly at the knots — $O(nm)$, or $O(n \log n)$ for the hard G by sorting the scores and taking a cumulative sum of τ ;
3. recover every t_j by interpolating G at s_j^* .

Remark 1 (the interpolant must be monotone). Use PCHIP, or linear interpolation on the monotone knot grid. A plain cubic spline and a GP posterior mean can both overshoot, and an overshoot destroys the implication in Proposition 1 — the whole point of the construction.

Cost falls from $O(n^2)$ to $O(n \log n + nm)$, which is the difference between a feature that can be recomputed on every forward pass and one that cannot.

Remark 2 (this is a different kernel from `anchor_interpolation.py`). There, the kernel lives in augmented block-geometry space $[z_i; \lambda c_i]$ and its job is to spread a constraint correction laterally across the deposit. Here it lives in one-dimensional score space and its job is to smooth a monotone reduction. Same word, unrelated operators. Do not reuse `build_features` or the Wendland radius for this.

4. What the feature buys, and the gradient it carries

Once t_j exists, everything downstream is differentiable in closed form. The period index $p_j = \lceil t_j \rceil$ softens to

$$p_j = \sum_k \sigma\left(\frac{t_j - k}{\varepsilon}\right), \quad \pi_{jk} = \sigma\left(\frac{t_j - k + 1}{\varepsilon}\right) - \sigma\left(\frac{t_j - k}{\varepsilon}\right),$$

the latter a soft one-hot over periods; and the discount is simply γ^{t_j} , with no capacity-crossing sigmoid stack, because t is *already* in period units.

The gradient is the informative part. With $\phi = \Phi'$,

$$\frac{\partial t_j}{\partial s_i^*} = -\frac{\tau_i}{h} \phi\left(\frac{s_i^* - s_j^*}{h}\right) \quad (i \neq j), \quad \frac{\partial t_j}{\partial s_j^*} = \frac{1}{h} \sum_i \tau_i \phi\left(\frac{s_i^* - s_j^*}{h}\right) = \hat{f}_\tau(s_j^*),$$

the τ -weighted kernel density of scores at s_j^* . Two consequences.

(i) **h is a receptive field in score space**, not a generic temperature. Only blocks within $O(h)$ of j in score exchange gradient. As $h \rightarrow 0$ the schedule becomes exact but the gradient collapses onto near-ties and training stalls; large h gives a dense but badly biased t . Anneal h as a quantile spacing — e.g. the score gap at the q -th percentile — so it stays scale-free as the score distribution drifts during training.

(ii) **Gradient magnitude tracks local score density**. A block in a crowded score region moves its own t hard; a score outlier is nearly frozen. If that shows up as pathological, normalise by $\hat{f}_\tau$ and accept the resulting bias.

5. The seam to settle before building

t is a function of s^* , which is the network’s output. Feeding t back in as an input feature is therefore a fixed-point equation $s^* = F(x, t(s^*))$, not a feature map. Three coherent readings, and they are materially different builds:

(a) **loss-side only.**

t never enters the encoder; replace the sigmoid stack in `soft_npv` with γ^{t_j} and stop there.

(b) **decoder feature, one unrolled pass.**

$s^{(1)} = F(x)$, then $t = \tau + G_h(s^{(1)})$, then $s^{(2)} = F(x, t)$, backpropagating through both passes. This is what “a feature based on the projected score” most naturally means.

(c) **target-BO seam.**

t enters `target_bo`’s `soft_profile` as a scheduled-time target rather than a score target.

My reading of the request is (b). Three things fix the operator completely, and I want them agreed in writing before code exists:

1. which of (a)/(b)/(c);
2. the tie-break rule — lexicographic or split-mass;
3. whether $\tau_j = \max_r w_{r,j}/C_r$ over all resources, or mining tonnage alone.

The last two times this operator was built from an unconfirmed reading, it was the wrong operator.